Formation 2 :

Premiers pas en ML

KRYPTOSPHERE®

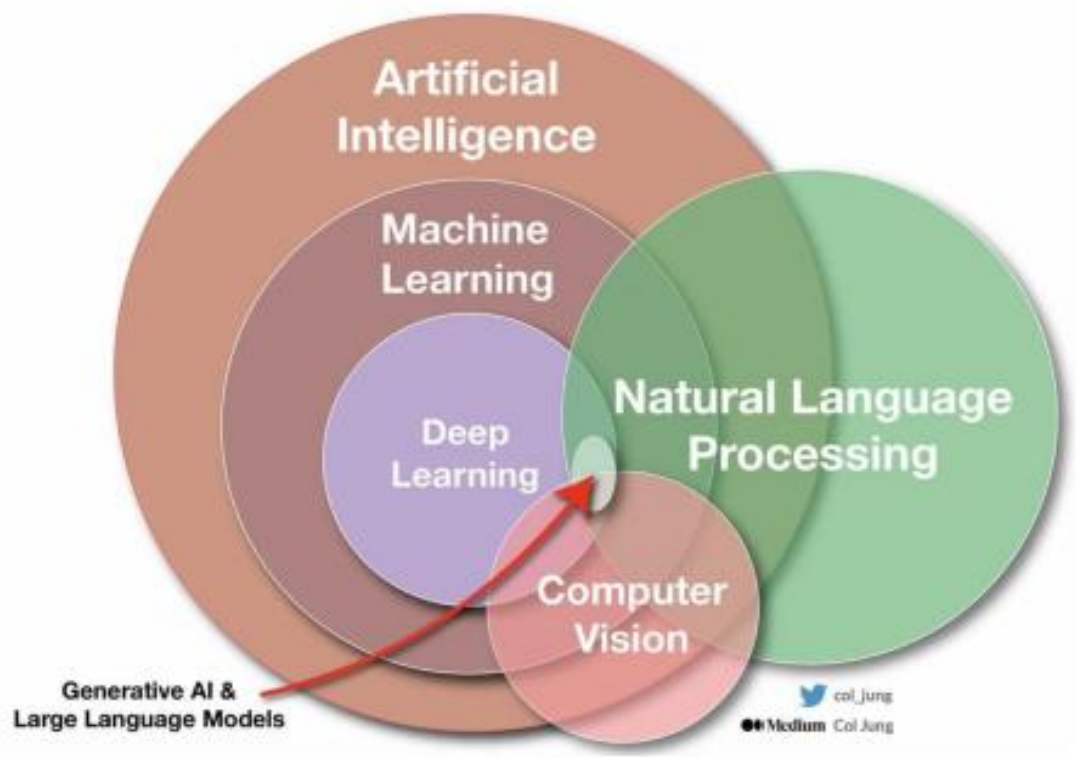
Plan :

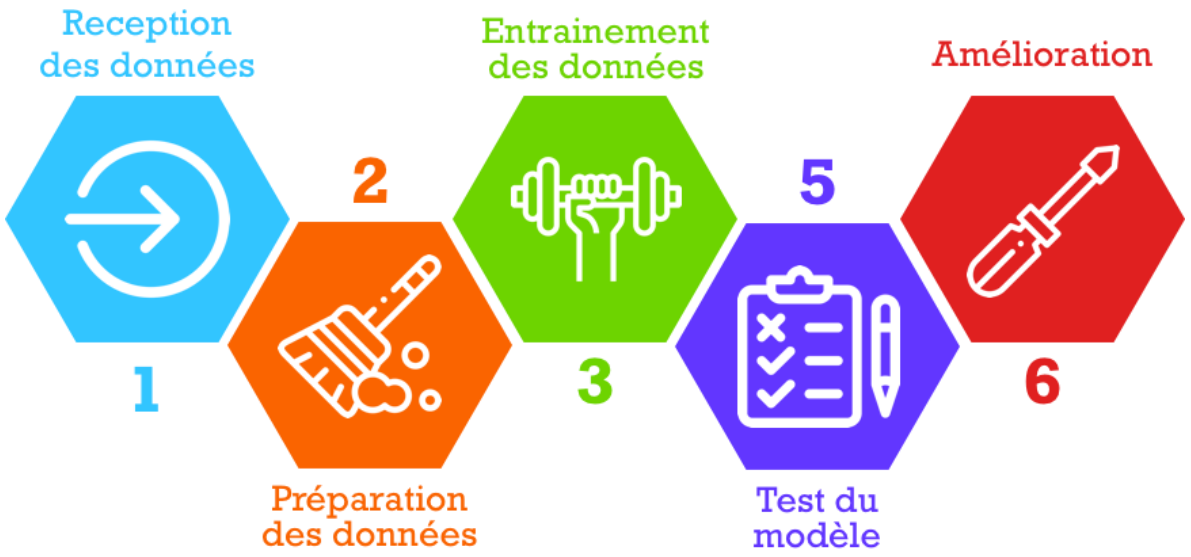
Régression et Classification ?

Manipuler les bases de données : pandas

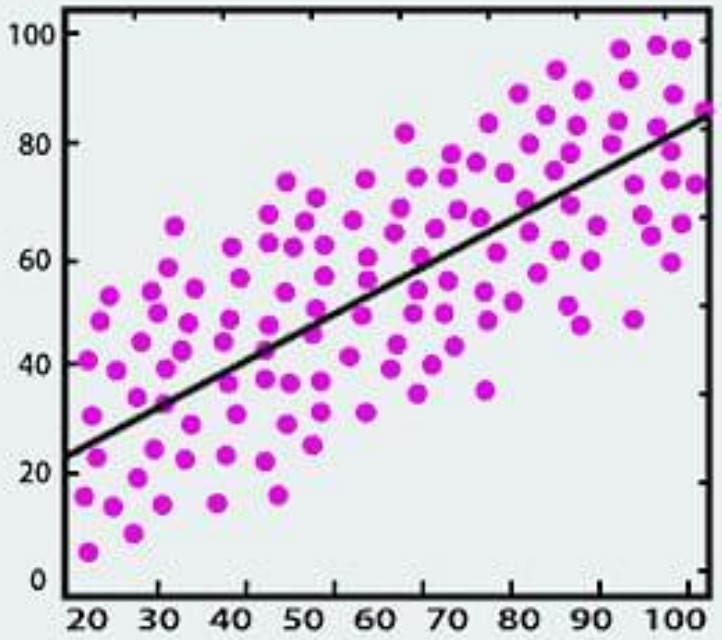
Le meilleur modèle : XGBoost

INTRODUCTION / Rappels



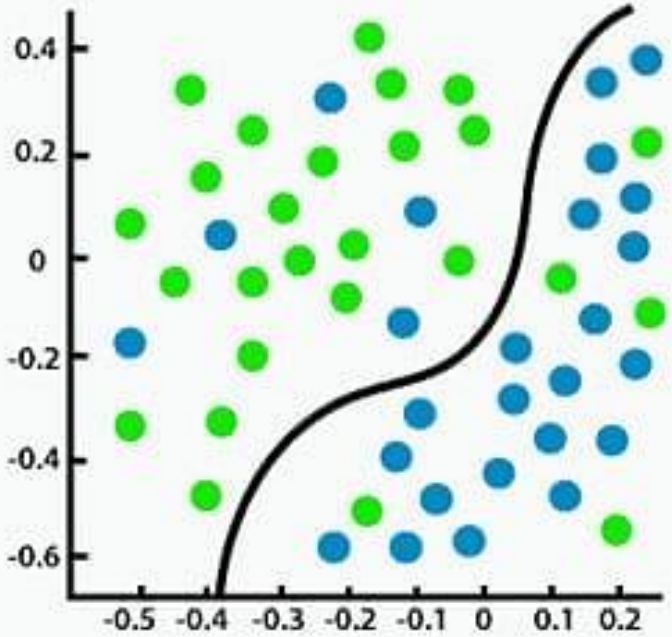


country	description	designation	points	price	province	region_1	region_2	taster_name	taster_twitter_handle	
Italy	Aromas include tropical fruit, broom, brimston...	Vulkà Bianco	87	NaN	Sicily & Sardinia	Etna	NaN	Kerin O'Keefe	@kerinokeefe	Nicosia 2013 Vu
Portugal	This is ripe and fruity, a wine that is smooth...	Avidagos	87	15.0	Douro	NaN	NaN	Roger Voss	@vossroger	Quinta dos Avid Avidagos Re
US	Tart and snappy, the flavors of lime flesh and...	NaN	87	14.0	Oregon	Willamette Valley	Willamette Valley	Paul Gregutt	@paulgwine	Rainstorm 2013 (Willame
US	Pineapple rind, lemon pith and orange blossom ...	Reserve Late Harvest	87	13.0	Michigan	Lake Michigan Shore	NaN	Alexander Peartree	NaN	St. Julian 2013 Re Harvest
US	Much like the regular bottling from 2012, this...	Vintner's Reserve Wild Child Block	87	65.0	Oregon	Willamette Valley	Willamette Valley	Paul Gregutt	@paulgwine	Sweet Ch Vintner's Re
Spain	Blackberry and raspberry aromas show a typical...	Ars In Vitro	87	15.0	Northern Spain	Navarra	NaN	Michael Schachner	@wineschach	Tandem 2011 A Tempranillo-M
Italy	Here's a bright, informal red that opens with ...	Belsito	87	16.0	Sicily & Sardinia	Vittoria	NaN	Kerin O'Keefe	@kerinokeefe	Terre di Giurfo 20 Frappato
France	This dry and restrained wine offers spice in p...	NaN	87	24.0	Alsace	Alsace	NaN	Roger Voss	@vossroger	Trim Gewurztramin
Germany	Savory dried thyme notes accent sunnier flavor...	Shine	87	12.0	Rheinhessen	NaN	NaN	Anna Lee C. Iijima	NaN	Heinz Eifel 2 Gewürztraminer
France	This has great depth of flavor with its fresh ...	Les Natures	87	27.0	Alsace	Alsace	NaN	Roger Voss	@vossroger	Jean-Baptiste A Les Natures P



Regression

versus



Classification

Manipuler les BDD en python : Pandas

- import pandas as pd
- df = pd.read_csv(“lien du fichier") -> mets la BDD dans df
- df.head(n) -> affiche les n premières lignes de la base de données
- df.info() -> colonne, nombre de non null, type
- df.describe(“include=all”) -> infos statstiques sur les colonnes
- df[‘colonne’] -> selection de la colonne
- df.drop(‘colonne’) -> supprime la colonne passée en argument

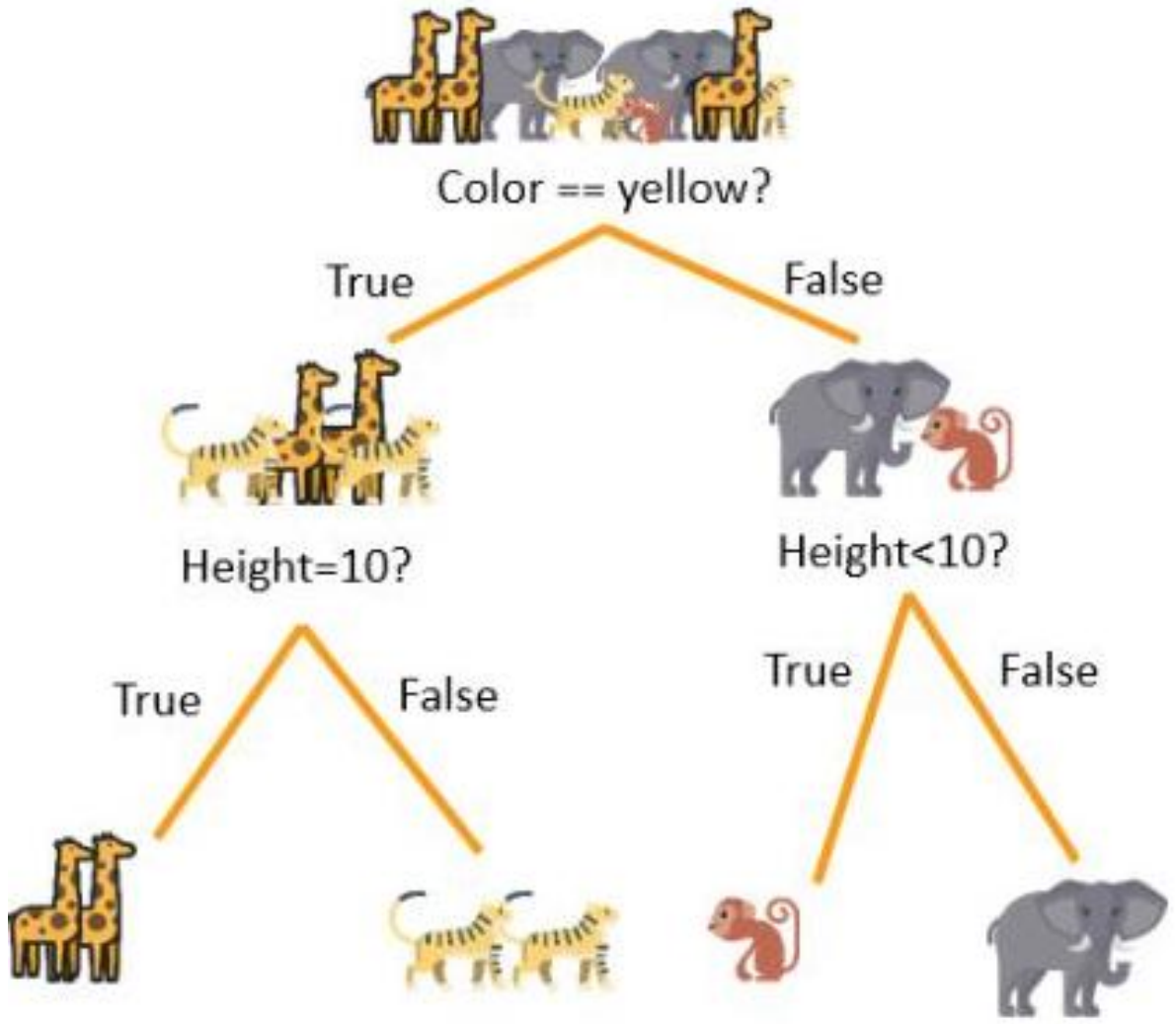
-> Premiers pas en data visualisation

Le meilleur modèle : XGBoost

Arbre de décision

Fonctionnement :

L'arbre est créé de haut en bas. A chaque création de nœud, on teste pour chaque variable et différents seuils quelle question crée des groupes avec le moins d'impureté.



Impureté d'un groupe

- Pour la classification :

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

$$Gini(E) = 1 - \sum_{j=1}^{c} p_j^2$$

Où les pi sont les proportions d'éléments de la classe i

Idéalement : 0

Pour la régression :

MSE = $\frac{1}{N_{node}} \sum_{i \in node} \left(y^{(i)} - \hat{y}_{node}\right)^2$

Avec $\hat{y}_{node}$ la moyenne des yi dans le nœud

Idéalement : 0

-> On cherche le split qui minimise l'impureté moyenne des 2 groupes créés.

Forêts de décisions

Défaut des arbres de décisions : sensibles à l'overfitting, instable

Solution : forêt de décision :

On sépare le dataset en bootstrap (échantillons avec remises), qui donnent chacun un arbre.

A chaque split on ne considère qu'une partie du boostrap.

Lors de la prédiction, on moyenne le résultat donné par chaque arbre.

XGBoost

- Les arbres ne sont plus indépendants, mais séquentiels, où chaque arbre cherche à corriger les erreurs du précédent.
- Il commence par initialiser les prédictions de façon quelconques (en général uniforme)
- Chaque nouvel arbre essaye de prédire les erreurs observer entre la prédiction actuelle et le dataset pour les corriger

XGBoost en pratique

- Import Xgboost as xgb
- model = xgb.XGBRegressor() ou model = xgb.XGBClassifier()
- model.fit(X_train, y_train)
- model.predict(x)

A vous de jouer !

TP : Boston Housing

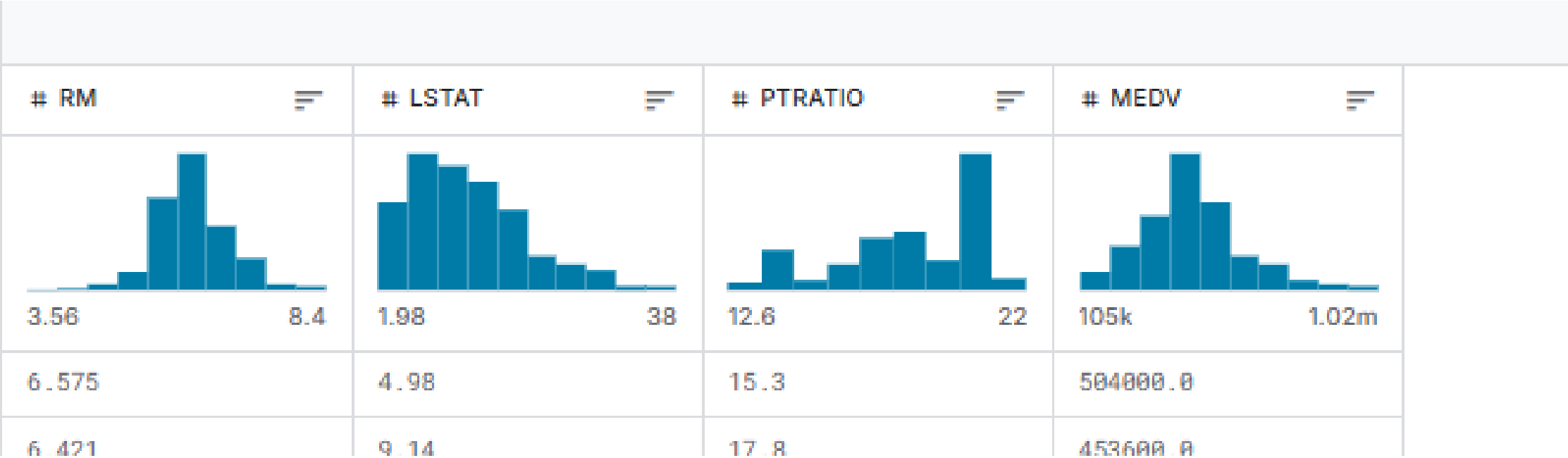
- Objectif : avoir un modèle XGBoost entrainer sur la dataset Boston Housing

- Premières étapes :

    1. Aller sur https://www.kaggle.com/datasets/schirmerchad/bostonhoustingmlnd

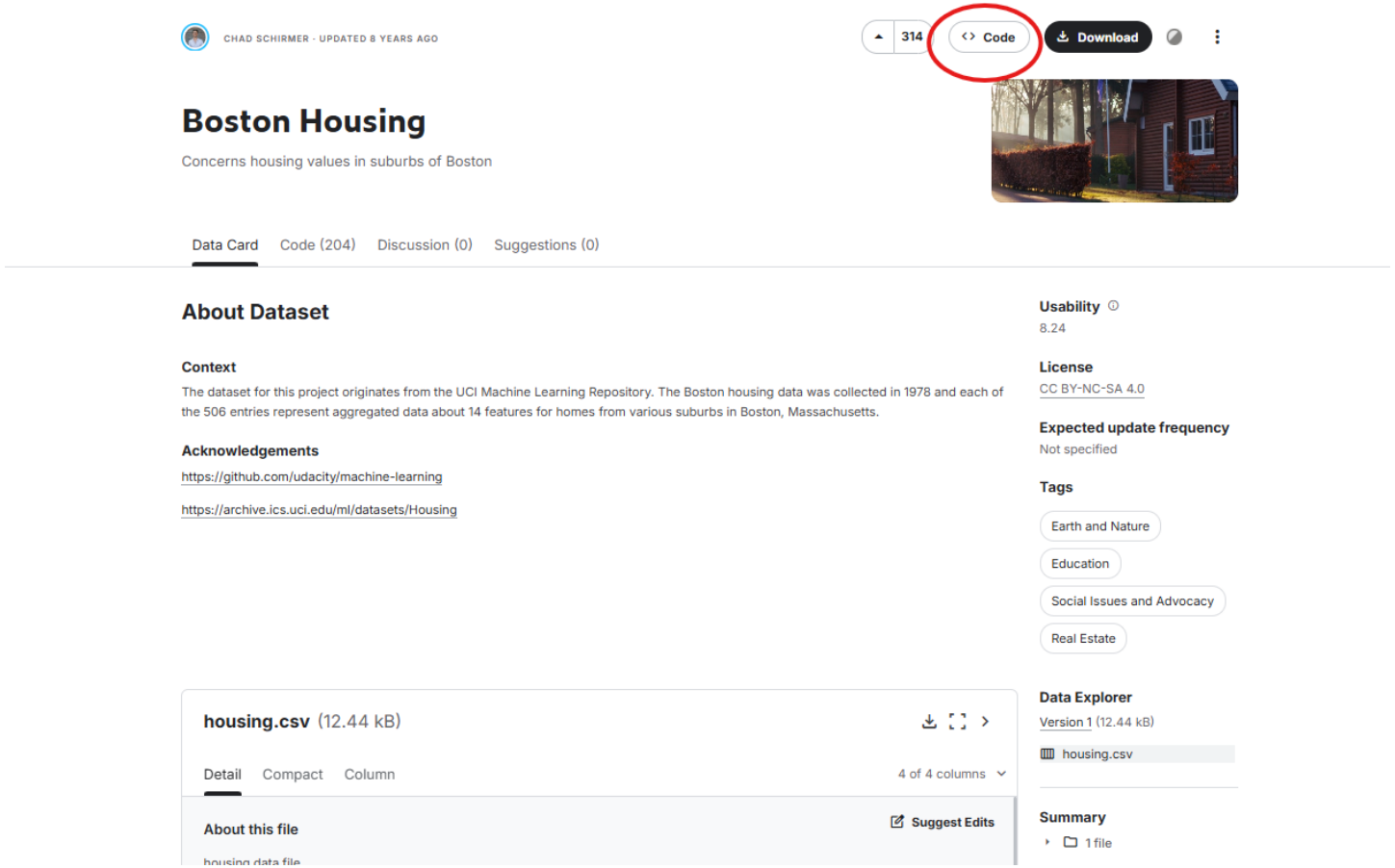
La base de Données : Boston Housing

- **RM** : Nombre moyen de pièces par logement .
- **LSTAT** : Pourcentage de la population appartenant à la classe socio-économique basse
- **PTRATIO** : Ratio élèves/enseignant dans les écoles de la ville
- **MEDV** : Valeur médiane des maisons occupées par leurs propriétaires. Variable à Prédire



# RM	# LSTAT	# PTRATIO	# MEDV
3.56 … 8.4	1.98 … 38	12.6 … 22	105k … 1.02m
6.575	4.98	15.3	504000.0
6.421	9.14	17.8	453600.0

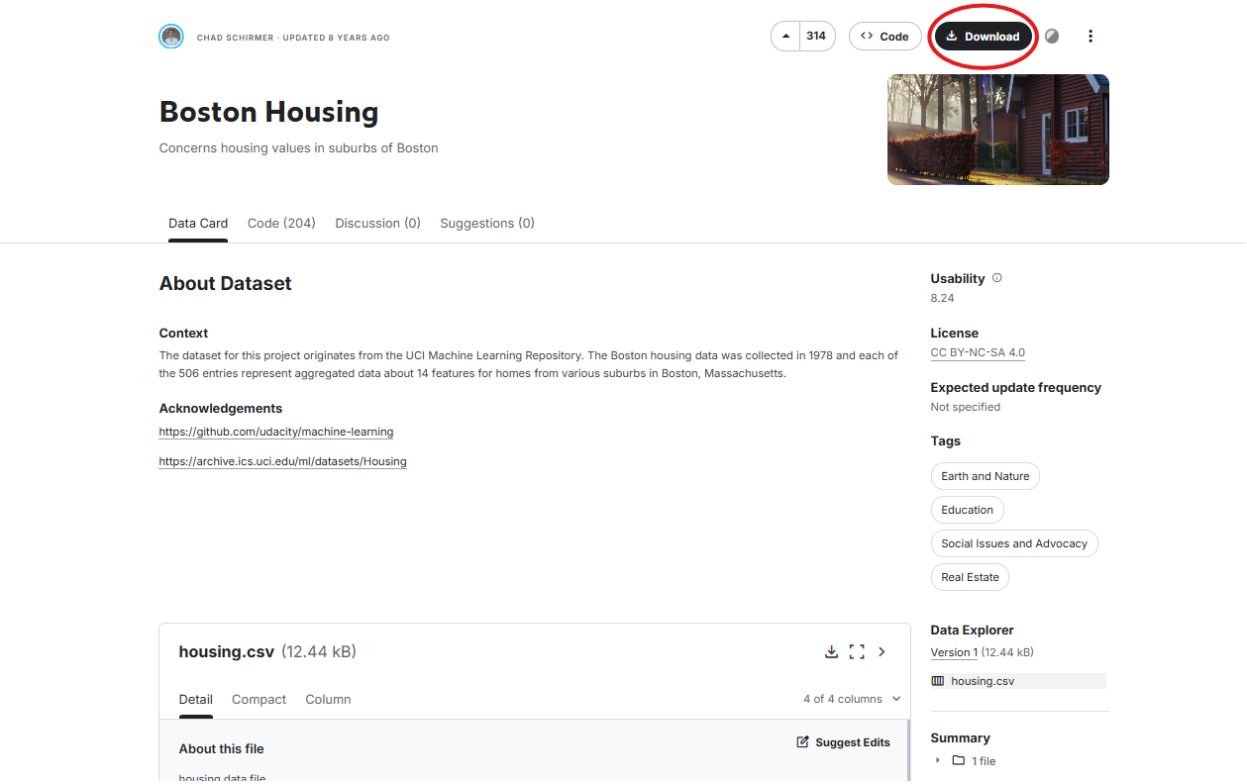
Si vous voulez coder sur kaggle :



- Ensuite, cliquer sur « create a notebook »

- La première ligne que vous devriez écrire est
- df = pd.read_csv("/kaggle/input/bostonhoustingmlnd/housing.csv")

Si vous voulez coder sur votre propre ide



- Télécharger la dataset dans le même répertoire que votre fichier .ipynb

- Utiliser pip install pour les bibliothèques dans un temrinal : pandas, XGBoost (je vous conseille d’utiliser un venv)